

Cele două aplicații cloud — exemplu gata de rulat

Ghid · proiecte-de-test/ · nginx pe host, o singură poartă

Nu e un exercițiu. E un exemplu care merge, verificat pe o instanță EC2 curată, pe care îl pornești în patru pași și apoi îl citești ca să-l poți repeta cu proiectele tale.

Trei fișiere: `compose-cloud-apps.yaml` (un MySQL + cele două aplicații, publicate **numai** pe `127.0.0.1:8120` și `8121`), `cloud-apps.conf` (o gazdă, două rute, o pagină de index) și `.env.example` (numele secretelor, fără valori).

Pornire — patru pași, pe server

1. Secretele se generează, nu se inventează.

```
cd proiecte-de-test
{ echo "MYSQL_ROOT_PASSWORD=$(openssl rand -base64 24)"
  echo "SCHOOL_JWT_SECRET=$(openssl rand -base64 48)"
  echo "SHOP_JWT_SECRET=$(openssl rand -base64 48)"; } > .env
chmod 600 .env
```

2. Containerele. `config -q` înainte de `up` : dacă fișierul e stricat, afli acum, nu după ce ai pornit jumătate din platformă.

```
docker compose -f compose-cloud-apps.yaml config -q
docker compose -f compose-cloud-apps.yaml up -d
docker compose -f compose-cloud-apps.yaml ps
```

3. Aplicațiile, înainte de nginx. Pe server, nu de pe laptop: `127.0.0.1` înseamnă mereu „mașina de pe care tastez”.

```
curl -s -o /dev/null -w '%{http_code}\n' http://127.0.0.1:8120/
curl -s -o /dev/null -w '%{http_code}\n' http://127.0.0.1:8121/
```

`404` e răspunsul bun: aplicația trăiește, dar rădăcina nu e a ei — stă pe prefix. `000` înseamnă că nu ascultă nimeni, deci nginx n-are ce proxa. Un JVM cu MySQL în spate are nevoie de zeci de secunde.

4. nginx. Pui IP-ul în locul marcatului, legi fișierul, testezi, reîncarci.

```
sed 's/IP_PUBLIC/3.88.114.102/g' cloud-apps.conf \
| sudo tee /etc/nginx/sites-available/cloud-apps >/dev/null
sudo ln -sf /etc/nginx/sites-available/cloud-apps \
    /etc/nginx/sites-enabled/cloud-apps
sudo nginx -t && sudo systemctl reload nginx
```

`nginx -t &&` nu e stil: dacă testul pică, reload-ul nu se execută și rămâi pe configurația care mergea. Și `reload`, nu `restart` : worker-ii vechi apucă să termine cererile în curs.

CRITERIU DE GATA

`http://cloud.IP.nip.io/` arată lista de linkuri, iar `/school-cloud/swagger-ui.html` și `/shop-cloud/swagger-ui.html` deschid Swagger. Toate pe portul 80, fără număr de port în adresă.

Atenție la un fals „gata”: dacă vezi „Welcome to nginx!”, configurația ta nu s-a aplicat — codul `200` vine de la site-ul implicit. Criteriul e conținutul, nu codul.

DOUĂ REGULI CARE NU SE NEGOCIAZĂ

`.env` nu se comite. Verifici cu `git status` înainte de `git add`, nu după.

În Security Group rămân două reguli: SSH 22 de la IP-ul tău, HTTP 80 de la oriunde. Porturile 8120, 8121 și 3306 n-au ce căuta acolo — aplicațiile ascultă pe loopback tocmai ca să nu poată fi atinse din afară.

De ce arată așa — și ce schimbi când îl refolosești

Decizie	Motivul, și ce ajustezi la tine
gazădă proprie, server_name cloud.IP.nip.io	ca să stea lângă platform.conf fără să se bată cu el: două fișiere cu default_server = nginx refuză să pornească
proxy_pass fără / la final	aplicațiile au SERVER_SERVLET_CONTEXT_PATH , deci așteaptă prefixul; cu slash l-ai tăia și ai primi 404 pe tot. Prefixul din compose și cel din location trebuie să fie identice — altfel pare vina proxy-ului
location = / , nu location /	doar rădăcina primește pagina de index; o rută greșită primește 404 cinstit, nu un 200 care te minte
un singur MySQL, două baze	school_cloud_db și shop_cloud_db , separate prin createDatabaseIfNotExist . Două servere de bază de date pentru două aplicații mici înseamnă doar de două ori mai multă memorie
porturi pe loopback	cele din stânga din ports : sunt cele din proxy_pass ; alege-le să nu se bată cu ce rulează deja
secretele din mediu	numele nu au convenție: school citește JWT_SECRET , shop APP_SECURITY_JWT_SECRET . Se caută în application.yaml , nu se ghicesc

Două lucruri erau stricate în cod, nu în configurație

REPARATE ÎN IMAGINILE DIN EXEMPLU — MERITĂ ÎNȚELESE

- 1. Adresa din Swagger era scrisă în cod.** La shop, OpenApiConfig.java declara servers("http://localhost:8082") , deci „Execute” trimitea cererile spre calculatorul celui care se uită, nu spre server. Scos: springdoc construiește singur adresa, din cerere și din anteturile X-Forwarded-* pe care i le dă nginx.
- 2. Orice eroare ieșea ca 401 .** Când o cerere se termină cu eroare, Spring o retrimite intern către /error ; dacă ruta aia nu e publică, răspunsul care ajunge la tine nu mai e eroarea adevărată. Se recunoaște după câmpul path , care spune /error în loc de ruta cerută. Reparat cu .dispatcherTypeMatchers(DispatcherType.ERROR).permitAll() . Verifică singur: /shop-cloud/v3/api-docs întoarce acum 404 cinstit — la shop calea e /api-docs , fără /v3 .

VERIFICAT PRIN RULARE, PE EC2 CURAT

Ubuntu 24.04 x86, imaginile trase de pe Docker Hub, totul prin nginx și prin nip.io real: la **ambele** aplicații — înregistrare 201 , autentificare cu token, apel pe rută protejată 200 , fără token 401 , Swagger 200 , adresa din spec = cea publică, rută inexistentă 404 . Din afară, 8120 , 8121 și 3306 — închise.